FleetCentric Research: Overview for the Team

Nov 4, 2026 · @hassan

In one minute

The research component tests whether FAST's shuttle routes can be planned, and re-planned when a road is blocked, on the cheapest kind of server: 512 MB of memory and one tenth of a CPU (Render's free tier). It is the FYP's research contribution, separate from the website.

- **Why it matters:** routes are made by hand today, and nothing replans them when a protest or flood blocks a road.
- **Where it lives:** the `research/` folder. Nothing in it changes the website, its backend or the database.
- **Status on 4 Nov 2026:** the optimizer works and beats the current routes by about 20 percent in a model. The first experiment is 7 of 50 runs done. Rerouting is not started.
- **How to look at it:** the Research Hub on the research laptop, or the shared page at https://hassanh5n.github.io/FAST-Transport/ (live once GitHub Pages is switched on).

The research questions

FYP-1 covers optimization and rerouting; FYP-2 adds real-world context and prediction later.

Phase	Part	Question	Status
FYP-1	Optimization	Can all-stop morning routes be planned from a travel-time table within free-tier limits, and are they better than the current hand-made routes?	In progress
FYP-1	Rerouting	When a road is blocked, which road should the bus take, and which stops can it skip because students can walk to a nearby point, for the least total delay?	Not started
FYP-2	Context	How much do real incidents slow shuttle routes, recorded per area on a hexagon map (H3)?	Later
FYP-2	Prediction	Does that context predict travel times better than the timetable and the route planner?	Later

The rerouting idea already has a name in the literature: the Shortest Covering Path Problem (Current, Pirkul and Rolland, 1994). No paper found does it for one shuttle mid-trip under a measured compute budget, which is the gap this research claims.

Words you will see

Word	Plain meaning
Contractor	A bus company. Jadoon and Nadeem each run their own buses and routes; each is planned separately.
Stop	A pickup point. There are 311, with a count of students at each.
Route	One bus's ordered list of stops, ending at campus by 08:00.
Travel-time matrix	A table of driving times from every stop to every other stop. The optimizer only reads this table, never the map.
OSRM	Free, open-source route planner that built the matrix from OpenStreetMap data. Times assume empty roads.
Optimizer	`optimize.py`. Uses Google OR-Tools to find routes with the lowest total student ride time.

Word	Plain meaning
Free-tier limits	512 MB memory, no swap, 0.1 CPU. We copy them on a laptop with Linux (WSL) and systemd, not on a real server.
Gap	How much worse a plan is than the best one found, in percent. Lower is better.
Student-minutes	Every student's ride time added up. The main score: lower is better.
H3	Uber's map of hexagon cells. Planned for marking blocked areas and narrowing the search, not yet used.
Covering path	A bus path that passes within walking distance of every stop, without stopping at all of them. The core idea of rerouting.

What has been done

The first phase, building the optimizer and testing it, is mostly built. Each step below feeds the next.

1. **Dataset (fall2026-v1).** 311 stops (Jadoon 171, Nadeem 143, 3 shared), 52 routes, 53 buses and 2,415 seats, taken from the real route sheets. There are 1,806 made-up students (seed 2026), because we cannot use real student data. It is frozen, so every run uses the same input.
2. **Road network.** We downloaded a Karachi OpenStreetMap extract and ran OSRM on it once, with Docker.
3. **Travel-time matrices.** OSRM gave us driving times between all stops: 172 × 172 for Jadoon and 144 × 144 for Nadeem. They are saved in the repo, so nobody needs Docker or OSRM again unless the stops change.
4. **Optimizer (`optimize.py`).** Reads the matrix and the student counts, then plans routes with the lowest total ride time. Rules: nobody rides over 90 minutes, buses reach campus by 08:00, each stop takes 60 s plus 5 s per student, and no bus goes over its seats.
5. **Experiment runner (`run_budget.py`).** Runs the optimizer many times with different time limits, inside the free-tier limits, and records memory, time and score for each run. If it stops, it picks up where it left off.
6. **Research Hub.** A local web page that runs everything with buttons, shows results as maps and charts, and publishes them.

7. **Shared page.** A read-only copy of the hub on GitHub Pages for teammates and the supervisor. It updates by itself after each publish.
8. **Logbook.** `LOGBOOK.md` records every run, decision and result, with the date. The experiment's pass/fail rules were written down before any run, so we cannot move the goalposts later.

First results

Early numbers look good, but they are not final. They come from a short test run and part of the main experiment.

Development run (60 seconds per contractor). The optimizer cut total student ride time compared with the current hand-made routes:

Contractor	Optimized (student-minutes)	Change vs current routes
Jadoon	31,492	−21.7%
Nadeem	29,807	−20.0%

Time-budget experiment (7 of 50 runs done). This asks: under free-tier limits, how long must the optimizer run to get within 5% of its best plan? We decided that 60 s and 5% would count as a pass before running anything.

Contractor (limited)	Gap at 10 s	Gap at 60 s
Jadoon	12.5%	1.6%
Nadeem	10.4%	0.2%

So far, 60 seconds already gets within 2% of the best, and memory stays well under 512 MB. That points to a pass, but it needs the rest of the runs to be sure.

Keep in mind:

- Students are made up, so the totals are estimates.
- OSRM times assume empty roads, with no Karachi traffic.
- The −20% compares against our model of the current routes, not real measured ride times.
- The limits copy the free tier on one laptop; a real cloud server may behave a little differently.

What needs to be done next

In order. Steps 1 and 2 close out optimization; the rest is the rerouting phase, which has not started.

#	Task	What it means	Good first task for a newcomer?
1	Finish the time-budget experiment	43 runs left, about 4 hours. In the hub: Run & publish, then Resume now or Schedule… overnight.	Yes. Only needs the setup guide.
2	Write up optimization	Add the final numbers and verdict to the logbook and the report chapter.	Yes.
3	Confirm the rerouting question	Check with the supervisor that the new rerouting question replaces the old SQ1/SQ2 wording in the proposal.	Yes. A meeting, no code.
4	Road-block test	Set a road's speed to 0 in OSRM and check that routes really go around it; time how long the update takes. Needs Docker and OSRM.	With help.
5	Build the rerouting pieces	A walking graph between stops, the covering-path solver, and H3 cells to mark blocked areas and narrow the search.	No. The main coding work.
6	Walking-radius sweep	Re-plan after a block with walking limits from 0 m to 800 m and measure the minutes saved.	After step 5.
7	Must-serve stops	Add stops that must be visited (accessibility, unsafe walking after dark) and measure the extra minutes.	After step 5.

FYP-2 (later): record real incidents on an H3 grid and test whether they predict travel times better than the timetable and OSRM.

Every new experiment shows up in the hub by adding one entry to `hub/experiments.json`. Write its pass/fail rule in the logbook before the first run.

Where everything lives

Everything is in the `research/` folder. It never touches the website code.

Path	What is there
`Research Hub.bat`	Double-click to open the hub.
`README.md`	Full technical setup and commands.
`LOGBOOK.md`	Dated record of every run and decision. Read this second, after this document.
`FleetCentric_Research_Setup_Guide.pdf`	Step-by-step setup for a new laptop.
`data/`	The frozen dataset and the travel-time matrices.
`optimize.py`, `run_budget.py`	The optimizer and the experiment runner.
`build_instance.py`, `build_matrix.py`	Rebuild the dataset and matrices. Only needed if stops change.
`results/`	Raw results of every run.
`hub/`	The hub: server, page and `experiments.json`.
`literature/`	Papers we build on.

Shared page: https://hassanh5n.github.io/FAST-Transport/ (live once GitHub Pages is switched on for the repo).

Rules for the team:

- Never edit the dataset or past results. Make a new version instead.
- Log every run in `LOGBOOK.md`, good or bad.
- Write an experiment's pass/fail rule before running it.
- Publish from the hub so the shared page stays current.